

ASSIGNMENT 2

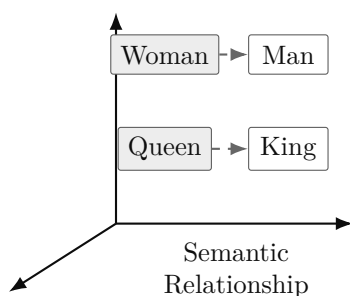
Design of a Vector Embedding for Capability Composition

Formal representations, compatibility, and compositional reasoning

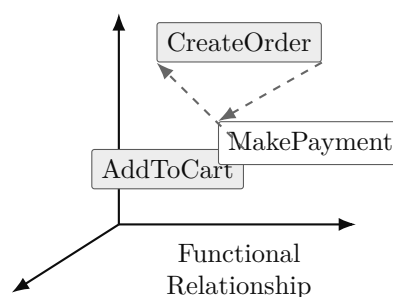
1 Motivation

In 2013, a team of researchers at Google including, **Tomáš Mikolov, Kai Chen, Greg Corrado, Ilya Sutskever, and Jeff Dean**, introduced Word2Vec and helped move modern natural-language processing from symbolic representations of words toward distributed vector representations. Word2Vec places words in a continuous vector space so that useful semantic relationships are preserved. Words with similar meanings or patterns of use lie close to one another, while algebraic relationships in the space can encode meaningful analogies.

**Word2Vec: Semantic relationships
between words**



**This Assignment: Functional
relationships
between capabilities**



While Word2Vec captures *semantic* relationships between words, this assignment focuses on *functional* relationships between capabilities in an application domain. Instead of meaning, the concern is which capabilities can work together, what they require, what they produce, and how they transform the state of a system.

Definition: Capability

A **capability** is a well-defined, reusable operation or service that performs a specific function within a system. It is characterized formally by what it requires (inputs, preconditions, and resources), what it does (effects), what it produces (outputs), and the constraints and quality attributes under which it operates.

Central task

Design, implement, and evaluate a problem-specific vector representation for formally specified states, goals, and executable capabilities. The representation should preserve the functional relationships required to identify compatible capabilities and construct complex capabilities from simpler ones.

2 Objective and Scope

In Assignment 1, you developed a planning and orchestration mechanism that searches for a sequence of transitions capable of transforming an initial state into a goal state while respecting constraints. This assignment changes the focus from *finding the sequence* to *representing the operations that make up the sequence*. These operations are called **capabilities**.

Your embedding should investigate relationships among:

states	goals	capabilities	preconditions
effects	inputs and outputs	constraints	resources
cost	reliability	availability	capability composition

The ultimate objective is to represent complex capabilities as compositions of smaller capabilities.

Important scope boundary

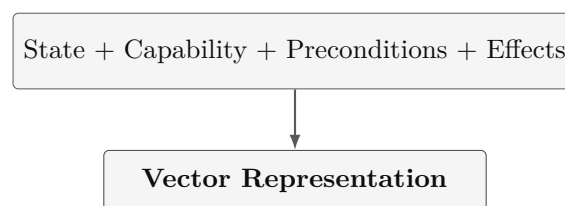
You are *not* required to redesign or improve the LPA*/D* Lite planner from Assignment 1. BFS, DFS, A*, D* Lite, LPA*, path search, and replanning algorithms are outside the scope of this assignment. The planner is background context only.

2.1 Relationship to Assignment 1

In Assignment 1, states and transitions were used to find a sequence of actions that transforms an initial state into a goal state. In this assignment, the focus changes from *finding the sequence* to *representing the operations that make up the sequence*.



Assignment 1: a sequence of transitions between states



Assignment 2: representing the operation itself

A transition in Assignment 1 can be viewed conceptually as an operation performed on a state. In Assignment 2, these operations are represented as capabilities. The planning/replanning algorithm from Assignment 1 remains outside the scope of this assignment; it is background context only.

3 Formal Application Model

A real-world application is represented formally as follows.

Formal application model

$$\mathcal{A} = (\mathcal{S}, \mathcal{C}, S_I, G, \mathcal{R}, \mathcal{K})$$

$\mathcal{S}$	State space of the application
$\mathcal{C}$	Set of available capabilities
S_I	Initial state
G	Goal specification
$\mathcal{R}$	Set of available resources
$\mathcal{K}$	Global constraints and policies

The initial state, goal, and capabilities are provided in formal rather than natural-language representations.

3.1 States and the initial state

A **state** describes the condition of the application at a particular point in time:

$$S = \{(x_1, v_1), (x_2, v_2), \dots, (x_n, v_n)\},$$

where x_i is a state variable and v_i is its current value. Variables may be Boolean, integer, real-valued, categorical, enumerated, or structured. The initial state $S_I \in \mathcal{S}$ specifies the condition before execution begins.

Example initial state

User.authenticated	true
User.role	CUSTOMER
Cart.exists	true
Cart.item_count	3
Order.exists	false
Payment.status	NOT_STARTED
Inventory.available	true
Notification.sent	false

3.2 Goal specification

A goal is a set of desired conditions,

$$G = \{g_1, g_2, \dots, g_m\}.$$

A state S satisfies the goal when $S \models G$.

Example goal specification

<code>Order.exists</code>	<code>true</code>
<code>Payment.status</code>	<code>SUCCESS</code>
<code>Notification.sent</code>	<code>true</code>

The overall application seeks a transformation

$$S_I \longrightarrow S_1 \longrightarrow \cdots \longrightarrow S_n \quad \text{such that} \quad S_n \models G.$$

Finding this sequence is a planning task and is not part of this assignment.

4 Formal Capability Model

A **capability** is a well-defined, reusable operation or service that performs a specific function within a system. It transforms application state and is characterized by what it requires, what it produces, how it changes the state, and the conditions under which it operates.

Formal capability representation

$$C_i = (T_i, I_i, O_i, P_i, E_i, K_i, R_i, Q_i, \text{Rel}_i, A_i, M_i)$$

Component	Meaning
T_i	Capability type
I_i, O_i	Input and output specifications
P_i, E_i	Preconditions and effects
K_i	Capability-specific constraints
R_i	Resource requirements
Q_i	Cost and other quality attributes
Rel_i	Reliability
A_i	Availability
M_i	Execution mechanism

This structure should contain enough information to reason about a capability without requiring a natural-language description.

4.1 Capability type and execution mechanism

The type identifies the broad mechanism through which a capability is realized:

$$T_i \in \{\text{API, DATABASE, GUI, EVENT, FUNCTION, FILE, COMPUTATION, MESSAGE, SERVICE}\}.$$

The execution mechanism M_i records implementation-specific details.

Examples of execution mechanisms

Type	Example mechanism
API	method = POST endpoint = /orders
Database	operation = INSERT table = orders
GUI	action = CLICK component = submit_button
Event	trigger = OrderCreated handler = SendNotification

Your design should determine how strongly type and mechanism should influence the embedding, and whether implementation-specific information belongs in the main vector or in a separate representation.

4.2 Inputs and outputs

A capability may require inputs and produce outputs.

Input specification

$$I_i = \{i_1, \dots, i_p\}$$

Each input is described separately:

$$i_j = (\text{name}, \text{type}, \text{domain}, \text{required}).$$

Example

$$i = (\text{cart_id}, \text{UUID}, \text{valid UUIDs}, \text{true}).$$

Output specification

$$O_i = \{o_1, \dots, o_q\}$$

Each output is described separately:

$$o_j = (\text{name}, \text{type}, \text{domain}).$$

Example

$$o = (\text{order_id}, \text{UUID}, \text{valid UUIDs}).$$

For `CreateOrder`, the examples above represent a required input and a produced output. If an output $o \in O_i$ satisfies an input requirement $i \in I_j$, then C_i may provide data required by C_j .

4.3 Preconditions and effects

The precondition set $P_i = \{p_1, \dots, p_r\}$ describes when a capability may run:

$$C_i \text{ is applicable to } S \iff S \models P_i.$$

Example preconditions

- `authenticated = true`
- `customer_exists = true`
- `cart_exists = true`
- `cart_item_count > 0`
- `inventory_available = true`

The effect set $E_i = \{e_1, \dots, e_s\}$ specifies the resulting state changes. If C_i is applicable, then

$$C_i : S \longrightarrow S', \quad S' = \text{Apply}(S, E_i).$$

Example effects

- `Order.exists = true`
- `Order.status = CREATED`
- `Cart.locked = true`

Example capability: CreateOrder

Preconditions	<code>CartExists = true</code> <code>CartItemCount > 0</code>
Effects	<code>OrderExists = true</code> <code>OrderStatus = CREATED</code>

$$\text{State } S \xrightarrow{\text{CreateOrder}} \text{State } S'.$$

4.4 Constraints and resources

Capability-specific constraints are represented as $K_i = \{k_1, \dots, k_t\}$. They may include input, state, resource, security, temporal, dependency, and domain-specific restrictions.

Example constraints

- `quantity > 0`
- `quantity <= inventory_available`
- `payment_amount <= transaction_limit`
- `user.role in {CUSTOMER, ADMIN}`

Resource requirements are represented as $R_i = \{r_1, \dots, r_u\}$.

Example resource requirements

- Database
- Payment gateway
- File system
- External service
- Authentication token
- Network
- GPU

Resource requirements must be considered in the capability representation.

4.5 Cost

A capability may have several operational costs or quality attributes:

$$Q_i = (C_{\text{time}}, C_{\text{resource}}, C_{\text{money}}, C_{\text{risk}}, C_{\text{energy}}).$$

Illustrative operational attributes

Execution time	100 ms
Monetary cost	0.01
Resource cost	2
Risk	0.1

You may introduce additional quality attributes where appropriate.

4.6 Reliability and Availability

Reliability. The reliability of a capability represents its probability of success:

$$\text{Rel}_i \in [0, 1].$$

For example, $\text{Rel}_i = 0.99$ represents an approximate success probability of 0.99 under the specified conditions. The embedding design should investigate whether reliability should be represented explicitly, indirectly, or separately from the semantic representation.

Availability. A capability may be available or unavailable at a particular point in time. A simple representation is

$$A_i \in \{0, 1\}.$$

For dynamic systems, availability may instead be represented as

$$A_i(t) \in \{0, 1\}.$$

For example, an external service may be available only during a specified period. The embedding design should investigate whether these operational properties should be embedded directly, represented separately, or handled through a combination of approaches.

5 Capability Composition

Consider two capabilities

$$C_1 : S_0 \rightarrow S_1, \quad C_2 : S_1 \rightarrow S_2.$$

If the outputs and effects of C_1 satisfy the inputs and preconditions of C_2 , then $C_2 \circ C_1$ is a valid composition. The resulting complex capability is

$$C_{12} = C_2 \circ C_1 : S_0 \rightarrow S_2.$$

Composition may be extended recursively, for example $C_{123} = C_3 \circ C_2 \circ C_1$. A central objective is to determine how such compositions should be represented in the proposed vector space.



Composite capability: CompletePurchase

CreateOrder produces the effect **OrderExists = true**.

This satisfies a precondition of **MakePayment**, which then produces **PaymentStatus = SUCCESS**.

Therefore,

$$\text{CompletePurchase} = \text{MakePayment} \circ \text{CreateOrder}.$$

6 Embedding Design Problem

Given $\mathcal{P} = (\mathcal{S}, G, \mathcal{C})$, design embeddings for the formal entities. You may use separate representations,

$$\phi_S : \mathcal{S} \rightarrow \mathbb{R}^{d_s}, \quad \phi_G : G \rightarrow \mathbb{R}^{d_g}, \quad \phi_C : \mathcal{C} \rightarrow \mathbb{R}^{d_c},$$

or a unified representation $\phi : (\mathcal{S}, G, \mathcal{C}) \rightarrow \mathbb{R}^d$. The choice is part of the assignment.

An embedding converts a formal entity into a numerical vector, for example

$$\text{formal capability} \xrightarrow{\phi_C} [0.21, -0.43, 0.71, \dots, 0.18].$$

Its purpose is not merely to assign a unique number. It should preserve useful relationships among capabilities, states, and goals.

6.1 Required properties

Your design must investigate the following properties.

1. **Capability identity.** Functionally different capabilities should be distinguishable.
2. **State awareness.** The representation should capture the relationship between a capability and the states on which it operates.

- 3. Precondition–effect compatibility.** It should capture relationships such as $E_i \Rightarrow P_j$, which may permit $C_i \rightarrow C_j$.
- 4. Input–output compatibility.** It should represent cases where one capability’s output satisfies another capability’s input requirements.
- 5. Similarity and composability.** Functional resemblance and composability are not the same. Similar capabilities need not compose; composition requires compatible outputs/effects and inputs/preconditions. For example, `CreateOrder` $\rightarrow$ `MakePayment` is compatible, whereas `CreateOrder` $\rightarrow$ `SendNotification` is not necessarily compatible. Explain how similarity is used when evaluating capability relationships.
- 6. Composition.** Provide a meaningful representation for $C_i \circ C_j$ and, where appropriate, $C_1 \circ C_2 \circ \dots \circ C_n$.
- 7. Goal relevance.** Allow the relationship between capabilities and the desired goal to be investigated.
- 8. Operational properties.** Determine how cost, reliability, availability, constraints, and resources should be embedded or represented separately.

No prescribed embedding technique

Study relevant representation and embedding methods; analyse their strengths and limitations; then design, justify, implement, and evaluate a solution specifically for capability composition. You may adapt or combine existing ideas, but merely applying an existing embedding model is not sufficient.

7 Required Experiments

Construct experiments using formally specified application problems.

1. Capability compatibility.

Given C_1, C_2, C_3 , where C_1 composes with C_2 but not C_3 , investigate whether the representation distinguishes the relationships. Use the following pattern:

Capability	Precondition	Effect
C_1 : CreateOrder	No additional precondition	<code>OrderExists = true</code>
C_2 : MakePayment	<code>OrderExists = true</code>	—
C_3 : CancelCart	<code>OrderExists = false</code>	—

Therefore, $C_1 \rightarrow C_2$ is compatible and $C_1 \rightarrow C_3$ is incompatible.

2. Capability composition.

Given $C_1 \rightarrow C_2 \rightarrow C_3$, construct the composite capability and investigate how its vector relates to the vectors of C_1, C_2, C_3 .

3. Alternative implementations.

Provide capabilities that produce similar effects through different mechanisms, such as an API, database, and GUI.

Test whether the embedding captures their functional relationship without incorrectly treating their implementations as identical.

4. Irrelevant capabilities.

Introduce capabilities that do not contribute to the specified goal. Test whether the representation distinguishes useful capabilities from irrelevant ones.

5. Operational attributes.

Investigate the influence of at least two of the following: cost, reliability, availability, risk, and resource requirement.

8 Evaluation Criteria

Evaluate the proposed embedding against every question below.

Property	Question to be evaluated
Capability representation	Can different capabilities be represented distinctly?
State relationship	Does the representation capture the relationship between capabilities and states?
Precondition–effect compatibility	Can composable capabilities be distinguished from incompatible ones?
Input–output compatibility	Can dependencies between capabilities be represented?
Composition	Can complex capabilities be represented from smaller capabilities?
Goal relevance	Can the relationship between capabilities and goals be identified?
Operational properties	Can cost, reliability, availability, and constraints be represented appropriately?
Consistency	Does the representation behave consistently across different problems?
Efficiency	What are the computational and storage requirements?

9 Deliverables

Submit all four components below.

Deliverable 1: Formal embedding design

Provide a mathematical specification covering:

- state, goal, and capability representations;
- the embedding function;
- a similarity and/or compatibility measure; and
- a composition operation, if applicable.

Deliverable 2: Implementation

Develop a working embedding system supporting suitable versions of:

<code>encode(state)</code>	Encode an application state.
<code>encode(goal)</code>	Encode a goal specification.
<code>encode(capability)</code>	Encode an atomic or composite capability.
<code>compose(capabilities)</code>	Construct a composite representation.
<code>similarity(x, y)</code>	Compare two encoded entities.

You may design the exact interface.

Deliverable 3: Experimental dataset

Construct or use formally specified application problems containing initial states, goal specifications, atomic capabilities, capability compositions, relevant constraints, and operational attributes.

Deliverable 4: Technical report

The report must cover:

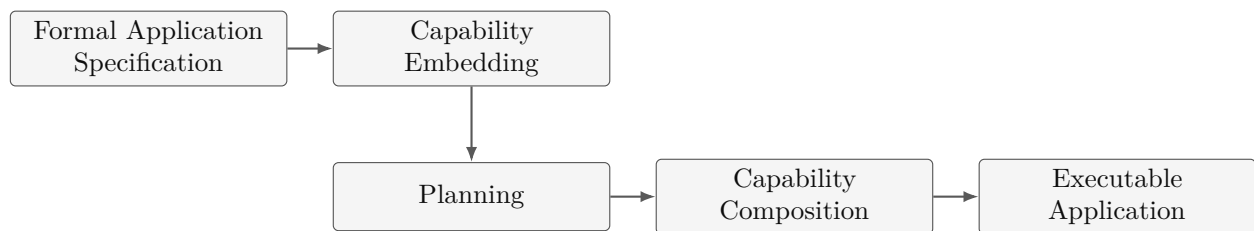
1. Problem definition
2. Design requirements
3. Related embedding approaches
4. Proposed representation
5. Mathematical formulation
6. Capability composition model
7. Implementation
8. Experimental methodology
9. Results
10. Analysis
11. Limitations
12. Conclusion

10 Final Objective

The ultimate objective is to determine whether a suitably designed vector representation can capture the structure of a formally specified application. In particular, investigate whether atomic capabilities, once embedded, can be composed into meaningful representations of composite capabilities:



More broadly, this assignment is one step toward an architecture in which formally specified applications are embedded, composed, and ultimately planned into executable systems:



The primary research question of this assignment is therefore:

Primary research question

How can formally specified states, goals, and executable capabilities be represented in a vector space such that the representation preserves the relationships required for capability compatibility, composition, and the construction of complex application functionality?